



**UNIVERSITÉ DE TUNIS EL MANAR**

**ÉCOLE NATIONALE D'INGÉNIEURS DE TUNIS**

Département des Technologies de l'Information et de la  
Communication

---

# **Classification de l'Occupation du Sol par Apprentissage Profond**

---

Elaboré par :

**Hamza Essoussi  
Meriem Gharsallah**

Classe :

**2 INFO 1**

2025 - 2026



# 1. Vue d'ensemble du projet

Ce projet a pour objectif de construire un pipeline complet de classification de couverture terrestre (land cover) à partir d'images satellitaires Sentinel-2, en utilisant un réseau de neurones convolutif de type U-Net. L'approche couvre l'intégralité du workflow : acquisition des données, labellisation, prétraitement, entraînement du modèle, évaluation et test de généralisation sur une zone géographique nouvelle.

## 1.1 Objectifs

- Collecter des images satellitaires multi-spectrales couvrant le nord de la Tunisie via Google Earth Engine.
- Labelliser manuellement les classes de couverture terrestre à l'aide de QGIS.
- Entraîner un modèle U-Net + ResNet34 pour la segmentation sémantique en 6 classes.
- Évaluer les performances sur un jeu de test indépendant.
- Tester la capacité de généralisation du modèle sur une image de l'Algérie (zone non vue à l'entraînement).

## 1.2 Classes de couverture terrestre

| ID | Classe       | Description                                           |
|----|--------------|-------------------------------------------------------|
| 0  | Urbain       | Zones construites, bâtiments, routes, infrastructures |
| 1  | Végétation   | Forêts, zones arbustives, espaces verts naturels      |
| 2  | Eau          | Mers, lacs, retenues d'eau, cours d'eau               |
| 3  | Sol nu       | Terres non cultivées, sols dénudés, zones rocheuses   |
| 4  | Agriculture  | Terres cultivées, périmètres irrigués, cultures       |
| 5  | Désert/Sable | Zones désertiques, dunes, sebkhas                     |

# 2. Outils et environnement de travail

## 2.1 Plateforme d'exécution — Kaggle

L'ensemble du pipeline d'entraînement a été exécuté sur la plateforme Kaggle, qui offre un accès gratuit à des GPU de type T4 et un environnement Python préconfiguré.

- GPU : NVIDIA Tesla T4 x2 (DataParallel activé)
- RAM : 16 Go
- Stockage : /kaggle/working/ (persistant entre cellules), /kaggle/input/ (datasets en lecture seule)
- Internet : activé pour l'authentification Google Earth Engine

## 2.2 Collecte des données — Google Earth Engine (GEE)

Google Earth Engine est une plateforme cloud de traitement géospatial. Elle a été utilisée pour accéder à la collection d'images satellites Sentinel-2, appliquer des filtres temporels et de qualité, calculer les indices spectraux, puis exporter le composite vers Google Drive.

- Collection utilisée : COPERNICUS/S2\_SR\_HARMONIZED (Level-2A, réflectance de surface)
- Filtres appliqués : zone géographique (filterBounds), période temporelle (filterDate), couverture nuageuse < 20 % (CLOUDY\_PIXEL\_PERCENTAGE)
- Masquage nuages : bande SCL (Scene Classification Layer) — exclusion des classes 3 (ombres), 8, 9 et 10 (nuages)
- Composite : médiane temporelle sur la période Avril–Septembre 2023

## 2.3 Labellisation — QGIS

La labellisation des données d'entraînement a été réalisée manuellement avec le logiciel QGIS (Quantum GIS), un SIG open source. L'image RGB de la zone d'étude a été chargée dans QGIS, puis des polygones de référence ont été dessinés à la main pour chacune des 6 classes, en s'appuyant sur l'interprétation visuelle de l'image satellite.

- Outil utilisé : QGIS 3.x avec plugin de création de couches vectorielles
- Format d'export : GeoJSON / Shapefile (.shp) avec un champ d'attribut de classe
- Ces fichiers de labels ont ensuite été rastérisés (rasterio) pour produire une carte de labels (label map) au même référentiel que l'image Sentinel-2

## 2.4 Librairies Python

| Librairie                         | Usage                                                                       |
|-----------------------------------|-----------------------------------------------------------------------------|
| rasterio                          | Lecture/écriture des images GeoTIFF multi-bandes, découpage en patches      |
| geopandas                         | Chargement et rastérisation des fichiers de labels vectoriels (GeoJSON/SHP) |
| PyTorch                           | Framework de deep learning — définition du modèle, entraînement, inférence  |
| segmentation-models-pytorch (smp) | Implémentation U-Net avec encodeur ResNet34 préentraîné sur ImageNet        |
| albumentations                    | Augmentation des données (flips, rotations, jitter colorimétrique)          |
| numpy / matplotlib                | Manipulation des tenseurs, visualisation des bandes et résultats            |
| scikit-learn                      | Métriques d'évaluation : accuracy, F1-score, matrice de confusion           |
| gdown                             | Téléchargement des fichiers GeoTIFF depuis Google Drive vers Kaggle         |
| earthengine-api (ee)              | Interface Python pour Google Earth Engine                                   |

### 3. Pipeline de données

#### 3.1 Étape 1 — Collecte de l'image Sentinel-2 (GEE)

La zone d'intérêt (AOI) couvre le nord de la Tunisie, délimitée par les coordonnées suivantes :

- Longitude : 8.0° E → 11.6° E
- Latitude : 36.0° N → 37.4° N
- Surface approximative : ~350 × 160 km

Les 6 bandes spectrales Sentinel-2 sélectionnées sont :

- B2 (Bleu, 490 nm), B3 (Vert, 560 nm), B4 (Rouge, 665 nm)
- B8 (PIR, 842 nm), B11 (SWIR1, 1610 nm), B12 (SWIR2, 2190 nm)

#### 3.2 Étape 2 — Calcul des indices spectraux

En complément des 6 bandes brutes, 4 indices spectraux ont été calculés pour enrichir la représentation et améliorer la discrimination entre classes :

| Indice | Formule                                   | Utilité                              |
|--------|-------------------------------------------|--------------------------------------|
| NDVI   | $(B8 - B4) / (B8 + B4)$                   | Densité et vitalité de la végétation |
| NDWI   | $(B3 - B8) / (B3 + B8)$                   | Détection des surfaces en eau        |
| NDBI   | $(B11 - B8) / (B11 + B8)$                 | Identification des zones bâties      |
| BSI    | $((B11+B4)-(B8+B2)) / ((B11+B4)+(B8+B2))$ | Sol nu, zones désertiques            |

L'image finale contient donc 10 bandes (6 spectrales + 4 indices), toutes normalisées en Float32. Elle est exportée au format GeoTIFF à une résolution de 20 m/pixel (taille ~2-3 Go).

### 3.3 Étape 3 — Preprocessing & normalisation

Avant l'entraînement, une normalisation statistique est appliquée sur chacune des 10 bandes :

- Calcul de la moyenne (mean) et de l'écart-type (std) sur l'ensemble des pixels valides ( $> 0$ )
- Normalisation z-score :  $(\text{valeur} - \text{mean}) / \text{std}$
- Les statistiques sont sauvegardées (means.npy, stds.npy) pour être réutilisées à l'inférence

### 3.4 Étape 4 — Génération des labels

Le fichier de labels vectoriels produit sous QGIS est rastérisé avec rasterio pour produire une carte de labels (label\_map) de même dimension que l'image Sentinel-2. Chaque pixel reçoit une valeur entière de 0 à 5 correspondant à l'une des 6 classes. Les pixels non annotés reçoivent la valeur 255 (ignore index) et sont exclus du calcul de la loss.

### 3.5 Étape 5 — Création des patches

Le modèle U-Net travaille sur des images de taille fixe. L'image complète (plusieurs milliers de pixels de côté) est donc découpée en patches carrés de  $256 \times 256$  pixels avec un recouvrement configurable. Seuls les patches contenant au moins une classe annotée sont conservés.

- Taille des patches :  $256 \times 256$  pixels
- Bandes : 10
- Split : 70 % entraînement / 15 % validation / 15 % test
- Augmentation : flips horizontaux/verticaux, rotation  $90^\circ$ , jitter colorimétrique (albumentations)

## 4. Architecture du modèle — U-Net + ResNet34

### 4.1 Choix de l'architecture

L'architecture U-Net a été choisie pour ses excellentes performances en segmentation sémantique d'images, en particulier dans les domaines où les données annotées sont limitées. Sa structure encodeur-décodeur avec connexions de saut (skip connections) permet de conserver à la fois le contexte global et les détails fins.

L'encodeur est remplacé par un ResNet34 préentraîné sur ImageNet (transfer learning), ce qui permet une meilleure extraction des features même avec un dataset relativement petit.

### 4.2 Configuration

| Paramètre          | Valeur                              |
|--------------------|-------------------------------------|
| Encodeur           | ResNet34 (préentraîné ImageNet)     |
| Canaux d'entrée    | 10 bandes                           |
| Canaux de sortie   | 6 classes                           |
| Taille des patches | 256 × 256 pixels                    |
| Batch size         | 8                                   |
| Epochs             | 50                                  |
| Learning rate      | 1e-4 (avec CosineAnnealingLR)       |
| Loss function      | CrossEntropyLoss (ignore_index=255) |
| Optimiseur         | Adam                                |
| DataParallel       | Oui (2 GPU T4)                      |

### 4.3 Entraînement

L'entraînement a été effectué sur 50 epochs avec sauvegarde automatique du meilleur modèle (critère : val loss minimale). Une sauvegarde de secours (backup) est effectuée toutes les 5 epochs. L'historique (loss, accuracy par epoch) est sauvegardé au format JSON.

## 5. Évaluation du modèle

## 5.1 Métriques utilisées

- Overall Accuracy : proportion de pixels correctement classifiés sur l'ensemble du test set
- F1-Score macro : moyenne du F1 par classe (sans pondération par la fréquence)
- F1-Score weighted : F1 moyen pondéré par le support de chaque classe
- Matrice de confusion : visualisation des erreurs de classification par classe (en %)
- Rapport de classification : précision, rappel et F1 par classe

## 5.2 Sorties produites

| Fichier              | Contenu                                                   |
|----------------------|-----------------------------------------------------------|
| preview_rgb.png      | Aperçu RGB de l'image satellite Tunisie Nord              |
| label_map.png        | Ground truth (labels QGIS rastérisés)                     |
| learning_curves.png  | Courbes de loss et d'accuracy (train/val)                 |
| confusion_matrix.png | Matrice de confusion sur le test set (%)                  |
| carte_finale.png     | Comparaison RGB / Ground Truth / Prédiction U-Net         |
| UNET_best.pth        | Poids du meilleur modèle (sauvegardé sur Kaggle Datasets) |
| means.npy / stds.npy | Statistiques de normalisation (réutilisées à l'inférence) |

## 6. Test de généralisation — Algérie

### 6.1 Objectif

Une fois le modèle entraîné sur la Tunisie, une étape de test de généralisation géographique a été conduite sur une image Sentinel-2 du nord de l'Algérie. L'objectif est d'évaluer si le modèle, entraîné sur une zone précise, est capable de produire des prédictions cohérentes sur une région géographiquement et climatiquement similaire, mais non vue lors de l'entraînement.

### 6.2 Collecte de l'image Algérie

Le même pipeline GEE a été utilisé pour collecter l'image de la zone nord-algérienne :

- Zone : nord de l'Algérie, de la frontière marocaine à la frontière tunisienne
- Coordonnées : longitude  $-2.0^{\circ} \rightarrow 8.5^{\circ}$  E, latitude  $35.5^{\circ} \rightarrow 37.2^{\circ}$  N ( $\sim 500 \times 200$  km)
- Période : Avril–Septembre 2023 (identique à la Tunisie pour comparaison équitable)
- Résolution : 20 m/pixel — mêmes 10 bandes que l'image d'entraînement
- Taille estimée : 2 à 3 Go

### 6.3 Pipeline d'inférence

Le notebook de test (land-cover-classification-3) charge le modèle entraîné depuis Kaggle Datasets et effectue la prédiction en mode patch par patch :

- Chargement du modèle (unet\_best.pth) avec suppression du préfixe « module. » ajouté par DataParallel
- Chargement des statistiques de normalisation (means.npy, stds.npy) issues de l'entraînement Tunisie
- Découpage de l'image en patches de  $128 \times 128$  pixels (taille réduite pour accélérer l'inférence)
- Normalisation z-score avec les statistiques Tunisie appliquées à l'image Algérie
- Inférence patch par patch avec torch.no\_grad()
- Reconstruction de la carte de prédiction complète

### 6.4 Visualisation des résultats

Le résultat final est présenté sous forme d'une visualisation côte à côte :

- Image satellite Algérie en RGB (couleurs naturelles)
- Carte de prédiction U-Net avec légende des 6 classes
- Distribution statistique des classes prédites (% de pixels par classe)

## 7. Structure des notebooks

### Notebook 1 — Collecte Sentinel-2 Tunisie

(land-cover-classification-1) — Étapes 1 & 2 : setup Kaggle + collecte GEE

| Cellule | Description                                                          |
|---------|----------------------------------------------------------------------|
| 1–2     | Installation des librairies, imports                                 |
| 3       | Authentification GEE via secrets Kaggle                              |
| 4–6     | Vérification GPU, structure des dossiers, variables globales         |
| 7       | Définition de l'AOI (zone Tunisie Nord) + visualisation              |
| 8       | Chargement et filtrage de la collection Sentinel-2                   |
| 9       | Composite médiane + calcul des 4 indices spectraux                   |
| 10–11   | Export vers Google Drive + suivi de l'export                         |
| 12–14   | Téléchargement vers Kaggle, vérification et visualisation de l'image |
| A1–A7   | Collecte de l'image Algérie Nord (même pipeline)                     |

## Notebook 2 — Entraînement U-Net

(land-cover-classification-2) — Étapes 3 → 7 : preprocessing, labels, patches, U-Net, évaluation

| Étape | Description                                                                          |
|-------|--------------------------------------------------------------------------------------|
| 3     | Vérification de l'image, calcul des stats mean/std, visualisation des bandes         |
| 4     | Chargement des labels QGIS, rastérisation, affichage de la distribution des classes  |
| 5     | Découpage en patches 256×256, split train/val/test, augmentation de données          |
| 6     | Définition du modèle U-Net+ResNet34, entraînement 50 epochs, courbes d'apprentissage |
| 7     | Évaluation sur test set, matrice de confusion, carte de classification finale        |

### Notebook 3 — Inférence Algérie

(land-cover-classification-3) — Test de généralisation sur image non vue à l'entraînement

| Opération            | Détail                                                              |
|----------------------|---------------------------------------------------------------------|
| Chargement du modèle | UNET_best.pth depuis Kaggle Datasets (avec correction DataParallel) |
| Normalisation        | means.npy / stds.npy issus de l'entraînement Tunisie                |
| Prédiction           | Patches 128×128, reconstruction de la carte complète                |
| Visualisation        | RGB vs prédiction, légende 6 classes, distribution                  |

## 8. Résumé du workflow complet

| # | Étape                  | Outil principal          | Sortie                                 |
|---|------------------------|--------------------------|----------------------------------------|
| 1 | Collecte image Tunisie | Google Earth Engine      | sentinel2_tunisie_nord_2023.tif        |
| 2 | Labellisation          | QGIS                     | labels.geojson / labels.shp            |
| 3 | Preprocessing          | rasterio, numpy          | means.npy, stds.npy                    |
| 4 | Rastérisation labels   | rasterio, geopandas      | label_map (numpy array)                |
| 5 | Génération patches     | rasterio, albumentations | patches images + masques               |
| 6 | Entraînement U-Net     | PyTorch, smp             | UNET_best.pth, history.json            |
| 7 | Évaluation             | scikit-learn, matplotlib | confusion_matrix.png, carte_finale.png |
| 8 | Collecte image Algérie | Google Earth Engine      | sentinel2_algerie_nord_2023.tif        |
| 9 | Inférence Algérie      | PyTorch, rasterio        | prediction_algerie.png                 |

Ce workflow constitue un pipeline de bout en bout, de la collecte de données brutes à la prédiction sur de nouvelles zones géographiques, démontrant la capacité de généralisation d'un modèle de deep learning pour la télédétection.

